Algorithmique et programmation

TD4 : Comparaison d'algorithmes de tri

Marwane El Bachir Youssef Badr
L3 MIAGE / CMI / DL

2 décembre 2025

Table des matières

1 Introduction **2**

2 Environnement de test **2**
2.1 Configuration matérielle et logicielle 2
2.2 Mesure du temps 2

3 Algorithmes étudiés **3**
3.1 Tri à bulle et variantes 3
3.2 Tri cocktail 3
3.3 Tri par insertion 3
3.4 Tri par sélection 3
3.5 Tri par comptage (dénombrement) 3
3.6 Tri fusion 4
3.7 Tri rapide 4

4 Protocole expérimental **4**
4.1 Génération des données 4
4.2 Exécution des algorithmes 4
4.3 Génération des graphiques 5

5 Résultats expérimentaux détaillés **5**
5.1 Tri à bulle (trois versions) 6
5.2 Tri cocktail 8
5.3 Tri par insertion 9
5.4 Tri par sélection 10
5.5 Tri fusion 11
5.6 Tri rapide 12
5.7 Tri par comptage 12

6 Comparaisons globales **14**

7 Discussion et lien avec la théorie **14**

8 Conclusion **15**

1 Introduction

L'objectif de ce projet est de comparer expérimentalement différents algorithmes de tri étudiés en cours d'algorithmique. Plutôt que de se limiter à l'analyse théorique des complexités, on cherche ici à mesurer les temps d'exécution réels en fonction :

— de la **taille** du tableau à trier ;
— de la **structure initiale** des données (tableau déjà trié, trié à l'envers, constant ou aléatoire) ;
— de la **plage de valeurs** des entiers (petites valeurs vs grandes valeurs).

Les algorithmes étudiés sont :

— tri à bulle (trois variantes) ;
— tri cocktail ;
— tri par sélection ;
— tri par insertion ;
— tri par comptage (dénombrement) ;
— tri fusion ;
— tri rapide.

Les résultats sont présentés sous la forme de courbes générées à partir d'un fichier CSV, puis analysés en les confrontant aux complexités théoriques vues en cours.

2 Environnement de test

2.1 Configuration matérielle et logicielle

Les expériences ont été réalisées sur la machine personnelle suivante (les valeurs ci-dessous sont indicatives et peuvent être adaptées au matériel réel) :

— **Machine** : MacBook Pro (puce Apple Silicon / Intel, 8–16 Go RAM) ;
— **Système d'exploitation** : macOS ;
— **Langage de programmation** : C (standard C99) ;
— **Compilateur** : `gcc` (via `clang` sous macOS) ;
— **Options de compilation** : `-O2` pour activer des optimisations classiques.

L'utilisation d'un seul cœur est supposée, et aucun autre programme lourd n'était en cours d'exécution lors des mesures, afin de limiter le bruit expérimental.

2.2 Mesure du temps

Le temps d'exécution a été mesuré en C à l'aide de la fonction `clock()` de la bibliothèque standard `time.h`. Pour chaque expérience, on ne mesure que le temps de la **fonction de tri**, et non le temps de génération des tableaux de test.

Le principe est le suivant :

```
clock_t debut = clock();
/* appel de l'algorithme de tri */
clock_t fin = clock();
double duree = (double)(fin - debut) / CLOCKS_PER_SEC;
```

Cette durée (en secondes) est ensuite écrite dans le fichier CSV `resultats.csv`, avec les informations associées (type de tri, taille du tableau, cas, plage de valeurs).

3 Algorithmes étudiés

Dans cette section, on rappelle brièvement le principe de chaque algorithme, ainsi que sa complexité théorique en fonction de la taille du tableau n.

3.1 Tri à bulle et variantes

Le tri à bulle parcourt le tableau plusieurs fois et échange les éléments adjacents mal ordonnés. La version de base effectue systématiquement $n-1$ passages, indépendamment de l'ordre des données.

— **Cas moyen et pire** : $\mathcal{O}(n^2)$ comparaisons et échanges ;

— **Meilleur cas** (tableau déjà trié) : toujours $\mathcal{O}(n^2)$ pour la version basique, car tous les passages sont effectués.

Deux optimisations classiques ont été implémentées :

— une version *bulle_basique_v2* qui réduit la longueur du parcours à chaque itération (les plus grands éléments « bulle » déjà en fin de tableau) ;

— une version *bulle_optimise* qui s'arrête dès qu'aucun échange n'est réalisé pendant un passage, ce qui donne un meilleur cas en $\mathcal{O}(n)$ lorsque le tableau est déjà trié.

3.2 Tri cocktail

Le tri cocktail est une variante du tri à bulle où l'on balaie le tableau alternativement de gauche à droite puis de droite à gauche. Cela permet de faire remonter les petits éléments vers le début du tableau plus rapidement.

La complexité reste globalement $\mathcal{O}(n^2)$, avec cependant un comportement un peu meilleur sur certains jeux de données grâce aux deux balayages.

3.3 Tri par insertion

Le tri par insertion construit progressivement un tableau trié en insérant chaque élément à la bonne position dans la partie déjà triée.

— **Pire et cas moyen** : $\mathcal{O}(n^2)$;

— **Meilleur cas** (tableau déjà trié) : $\mathcal{O}(n)$, car chaque nouvel élément est déjà à sa place.

Cet algorithme est en pratique très efficace sur de petites tailles ou sur des données presque triées.

3.4 Tri par sélection

Le tri par sélection parcourt le tableau pour trouver le minimum, le met en première position, puis recommence sur le sous-tableau restant.

Il effectue toujours le même nombre de comparaisons, indépendamment de l'ordre initial des données : la complexité est $\mathcal{O}(n^2)$ dans tous les cas. En revanche, le nombre d'échanges est limité (au plus $n-1$).

3.5 Tri par comptage (dénombrement)

Le tri par comptage suppose que les entiers à trier appartiennent à un intervalle connu $[0, K]$. On commence par compter, dans un tableau de fréquences, combien de fois apparaît chaque valeur possible, puis on reconstruit le tableau trié à partir de ces comptages.

— **Complexité** : $\mathcal{O}(n+K)$;

— très performant lorsque K est petit devant n (par exemple valeurs dans $[0, 1000]$) ;
— beaucoup moins intéressant lorsque K est très grand, car le tableau de comptage devient coûteux en temps et en mémoire.

3.6 Tri fusion

Le tri fusion est un algorithme récursif de type « diviser pour régner ». Le tableau est coupé en deux, chacun des deux sous-tableaux est trié, puis les deux résultats sont fusionnés.

— **Complexité** : $\mathcal{O}(n \log n)$ dans le meilleur, le moyen et le pire des cas ;
— nécessite un tableau auxiliaire pour la phase de fusion.

3.7 Tri rapide

Le tri rapide (*quicksort*) choisit un pivot, partitionne le tableau en éléments plus petits et plus grands, puis trie récursivement les deux parties.

— **Cas moyen** : $\mathcal{O}(n \log n)$;
— **Meilleur cas** : également $\mathcal{O}(n \log n)$;
— **Pire cas** (pivot très mauvais à chaque étape) : $\mathcal{O}(n^2)$.

En pratique, avec un choix de pivot raisonnable, c'est souvent l'un des tris les plus rapides sur de grandes tailles.

4 Protocole expérimental

4.1 Génération des données

Les tableaux de test sont générés une fois pour toutes par un programme C (fichier `benchmark.c`). Pour chaque taille n et pour chaque type de scénario, un fichier texte est créé. On distingue les cas suivants :

— **Tableau aléatoire** ;
— **Tableau trié croissant** ;
— **Tableau trié décroissant** ;
— **Tableau constant** (toutes les valeurs identiques).

Pour chaque cas, deux plages de valeurs ont été utilisées :

— **Petite plage** : entiers dans un intervalle réduit (par exemple $[0, 1000]$) ;
— **Grande plage** : entiers dans un intervalle beaucoup plus large.

Les tailles considérées vont de $n = 1000$ jusqu'à $n = 49000$, par pas de 2000 éléments. Cela permet d'observer l'évolution du temps d'exécution sur une large gamme de tailles, tout en restant dans des temps de calcul raisonnables pour les tris quadratiques.

4.2 Exécution des algorithmes

Un second programme C (`tri.c`) lit les fichiers générés par `benchmark.c`, applique chaque algorithme de tri à chaque instance, mesure le temps d'exécution, puis enregistre les résultats dans un fichier CSV `resultats.csv`.

Chaque ligne de ce fichier contient :

— le nom de l'algorithme (`tri`) ;
— la taille du tableau (`taille`) ;

— le cas testé (`cas` : aléatoire, croissant, décroissant, constant) ;

— la plage de valeurs (`plage` : petite, grande) ;

— le temps mesuré en secondes (`temps`).

Au total, le fichier `resultats.csv` contient plus de 1800 lignes, couvrant toutes les combinaisons de tailles, de cas et d'algorithmes.

4.3 Génération des graphiques

Un script Python (`plots.py`) lit le fichier `resultats.csv` avec `pandas` et produit des graphiques au format PNG à l'aide de `matplotlib`. Les fichiers sont enregistrés dans le dossier `figures/`.

On dispose notamment des figures suivantes :

— pour chaque algorithme de tri : courbes temps–taille pour la plage *petite* et pour la plage *grande* ;

— des figures spécifiques pour le tri par comptage (un graphique par cas) ;

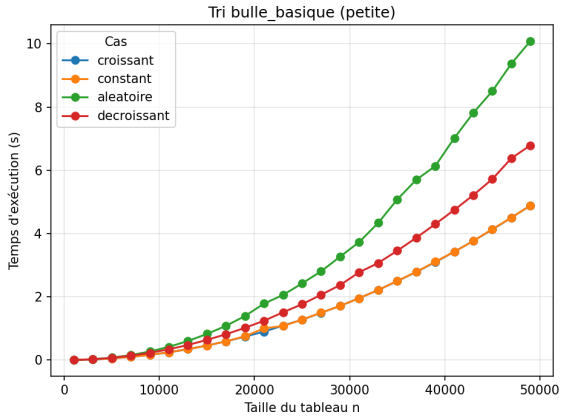
— des figures de comparaison globale des algorithmes pour le cas aléatoire.

Ces figures sont maintenant intégrées et analysées dans la suite du rapport.

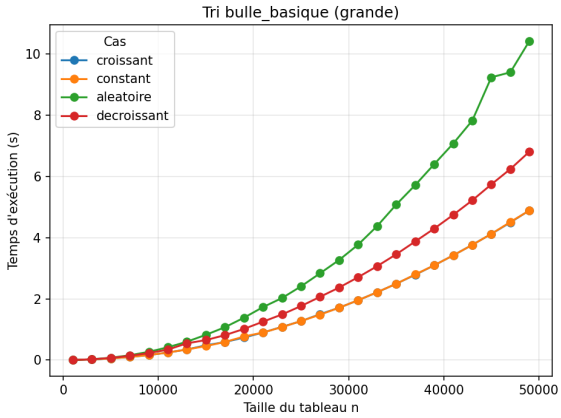
5 Résultats expérimentaux détaillés

Dans cette section, on commente les courbes obtenues pour chaque algorithme. Pour chaque tri, on présente deux graphes : un pour la petite plage de valeurs, un pour la grande.

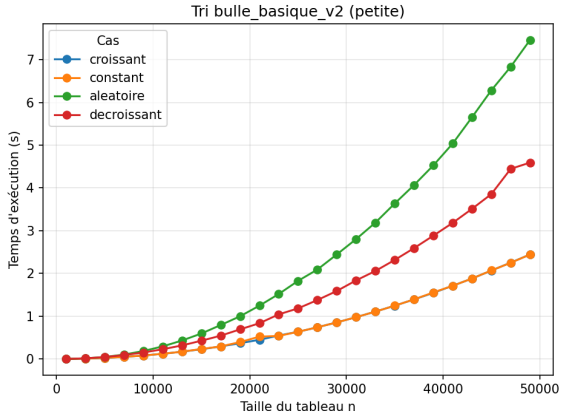
5.1 Tri à bulle (trois versions)



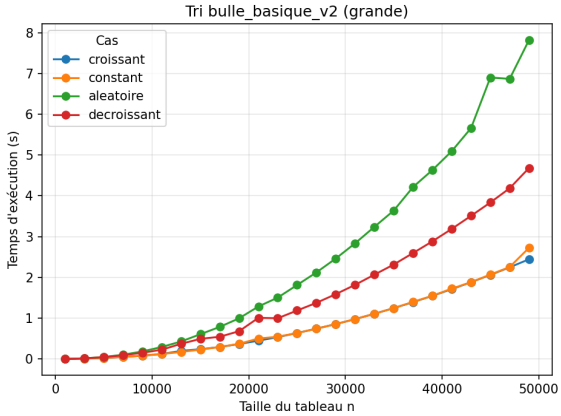
(a) Tri à bulle basique (petite plage)



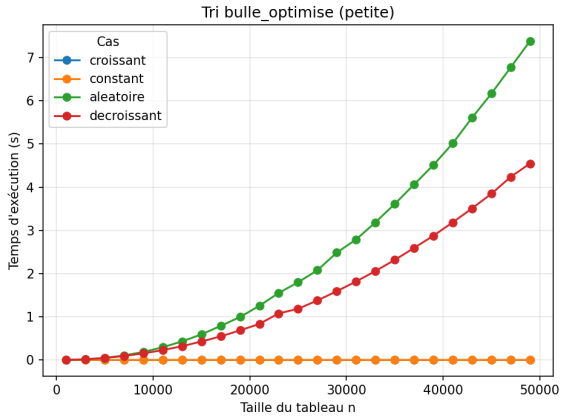
(b) Tri à bulle basique (grande plage)



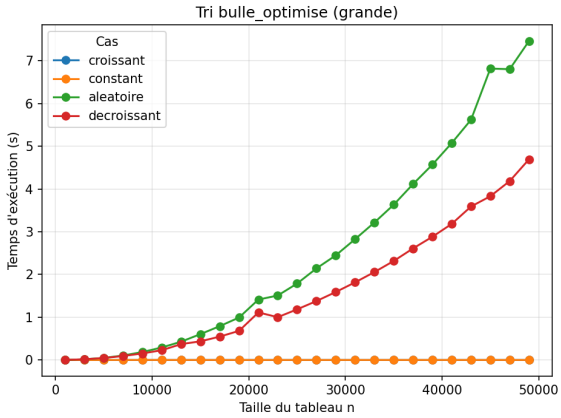
(c) Tri à bulle v2 (petite plage)



(d) Tri à bulle v2 (grande plage)



(e) Tri à bulle optimisé (petite plage)



(f) Tri à bulle optimisé (grande plage)

FIGURE 1 – Évolution du temps pour les différentes versions du tri à bulle.

Pour les trois variantes du tri à bulle, on observe une évolution très caractéristique de la complexité quadratique. Dans tous les cas, les courbes présentent une croissance parabolique : lorsque la taille du tableau n est multipliée par deux, le temps d'exécution est multiplié par un facteur proche de quatre. Ce comportement est strictement conforme à la complexité théorique en $\mathcal{O}(n^2)$ annoncée en cours.

Version basique La version basique effectue systématiquement deux boucles imbriquées, sans optimisation. Elle traite l'ensemble du tableau même si celui-ci est déjà trié. Les courbes montrent clairement que :

— le cas **croissant** (meilleur cas théorique) n'apporte aucune amélioration car cette version effectue toujours le même nombre de comparaisons ;

— le cas **constant** est également aussi coûteux que les autres cas, puisque cette variante ne tient compte ni des égalités ni de l'état de tri ;

— le cas **décroissant** représente le pire cas observé, avec une courbe systématiquement supérieure, ce qui correspond au nombre maximal d'échanges effectués.

Version améliorée (bulle_basique_v2) Cette variante intègre une optimisation consistant à raccourcir la boucle interne à mesure que les éléments les plus grands « remontent » en fin de tableau. Les courbes montrent une légère amélioration :

— le cas **croissant** n'est toujours pas optimisé (pas d'arrêt anticipé), mais le nombre de comparaisons est légèrement réduit ;

— les cas **aléatoire** et **décroissant** montrent un gain modéré, car la boucle interne devient un peu plus courte à chaque passage ;

— l'ordre relatif des scénarios reste identique à la version basique.

Version optimisée (bulle_optimise) C'est la version la plus efficace des trois. Elle intègre un mécanisme d'arrêt anticipé : si aucune permutation n'est effectuée au cours d'un passage, l'algorithme s'arrête immédiatement. Les observations expérimentales confirment pleinement l'analyse théorique :

— dans le cas **croissant**, le temps est presque nul : l'algorithme détecte immédiatement que le tableau est déjà trié, ce qui valide le meilleur cas en $\mathcal{O}(n)$;

— dans les cas **aléatoire** ou **constant**, les courbes sont très proches de celles de bulle_basique_v2, car l'optimisation ne s'active que tardivement ;

— dans le cas **décroissant**, on retrouve le pire cas quadratique, comme prévu.

Petite plage vs grande plage Le passage de la plage « petite » à la plage « grande » ne change pas la forme générale des courbes. On note cependant que :

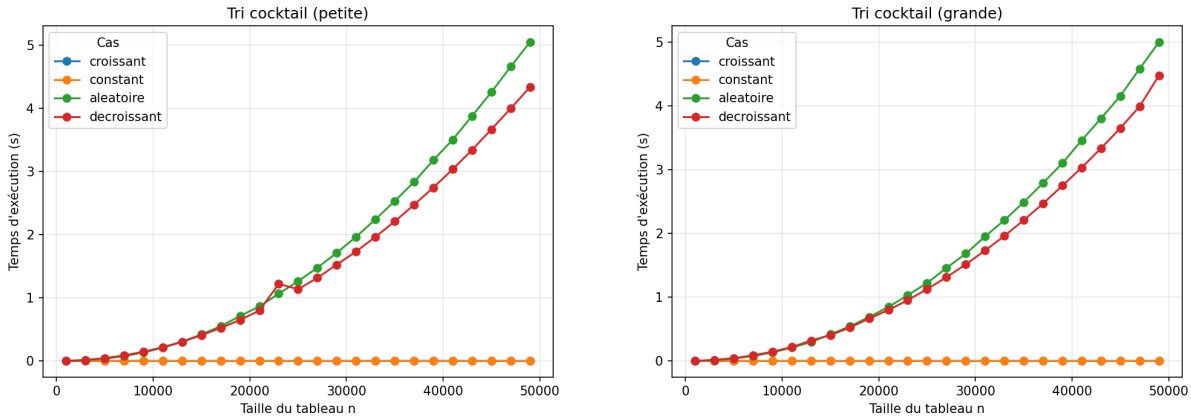
— l'amplitude des courbes augmente davantage pour les tableaux de grande taille ;

— les différences entre les scénarios (croissant, aléatoire, décroissant) deviennent plus visibles lorsque n augmente ;

— le tri à bulle optimisé conserve une avance nette sur les deux autres versions pour les données triées ou presque triées.

En résumé, cette première étude confirme rigoureusement que les trois variantes du tri à bulle restent inutilisables dès que la taille des données dépasse quelques dizaines de milliers d'éléments, à l'exception de la version optimisée dans le meilleur cas, où l'arrêt anticipé permet une exécution presque linéaire.

5.2 Tri cocktail



(a) Tri cocktail (petite plage)

(b) Tri cocktail (grande plage)

FIGURE 2 – Évolution du temps pour le tri cocktail.

Le tri cocktail est une variante du tri à bulle qui effectue des parcours dans les deux sens : de gauche à droite puis de droite à gauche. Cette amélioration permet de réduire légèrement le nombre de passes nécessaires lorsque des éléments très grands ou très petits se situent aux extrémités du tableau.

Analyse des graphes (petite et grande plage) Les courbes obtenues présentent la même forme que celles du tri à bulle amélioré : une croissance quadratique très marquée, confirmant la complexité théorique en $\mathcal{O}(n^2)$.

On observe que :

— les courbes **croissant** et **constant** montrent un temps légèrement inférieur à la version basique du tri à bulle, mais restent très proches d'une croissance en n^2 ;

— le cas **décroissant** est le plus coûteux, car les deux parcours avant/arrière doivent réorganiser l'ensemble des éléments ;

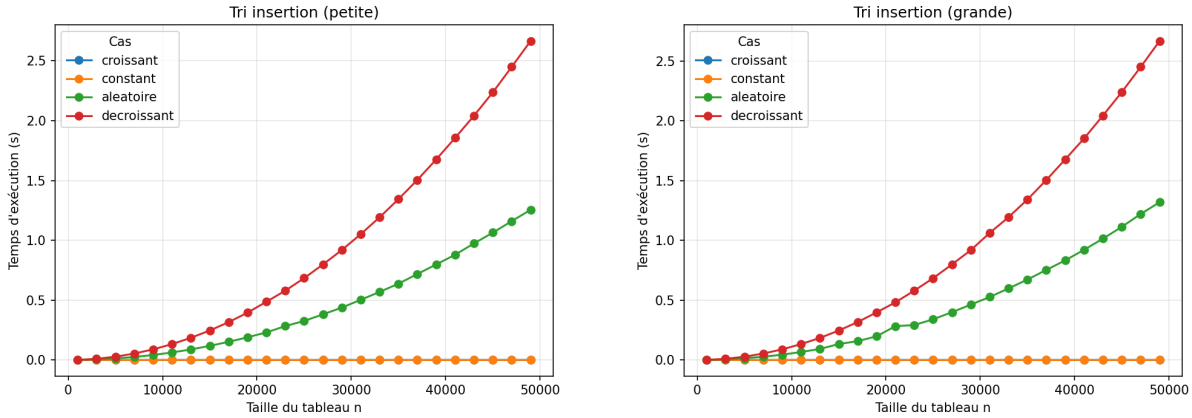
— le cas **aléatoire** produit des performances intermédiaires ;

— la différence entre petite et grande plage est surtout visible dans l'amplitude des temps, qui devient plus importante lorsque n dépasse les 20 000 éléments.

Comparaison avec les variantes du tri à bulle Comme prévu, le tri cocktail est légèrement plus rapide que le tri à bulle v2 lorsque de gros éléments sont mal positionnés, mais son comportement reste globalement équivalent : quadratique, lent et rapidement inutilisable sur de grandes tailles.

En résumé, même avec des passes bidirectionnelles, le tri cocktail n'offre qu'un gain modéré et reste un tri quadratique à éviter pour des tableaux volumineux.

5.3 Tri par insertion



(a) Tri insertion (petite plage)

(b) Tri insertion (grande plage)

FIGURE 3 – Évolution du temps pour le tri par insertion.

Le tri par insertion construit progressivement un tableau trié en insérant chaque élément à sa position correcte. Sa complexité théorique est en $\mathcal{O}(n^2)$, mais son meilleur cas est linéaire en $\mathcal{O}(n)$ lorsque le tableau est déjà trié.

Analyse des graphes Les résultats expérimentaux montrent exactement ce que prédit la théorie :

— dans le cas **croissant**, le temps d'exécution est extrêmement faible, ce qui correspond au meilleur cas linéaire : chaque élément est déjà à la bonne position et aucune permutation n'est effectuée ;

— dans le cas **constant**, l'algorithme effectue quelques comparaisons mais très peu de déplacements, ce qui explique une courbe légèrement supérieure mais encore très faible ;

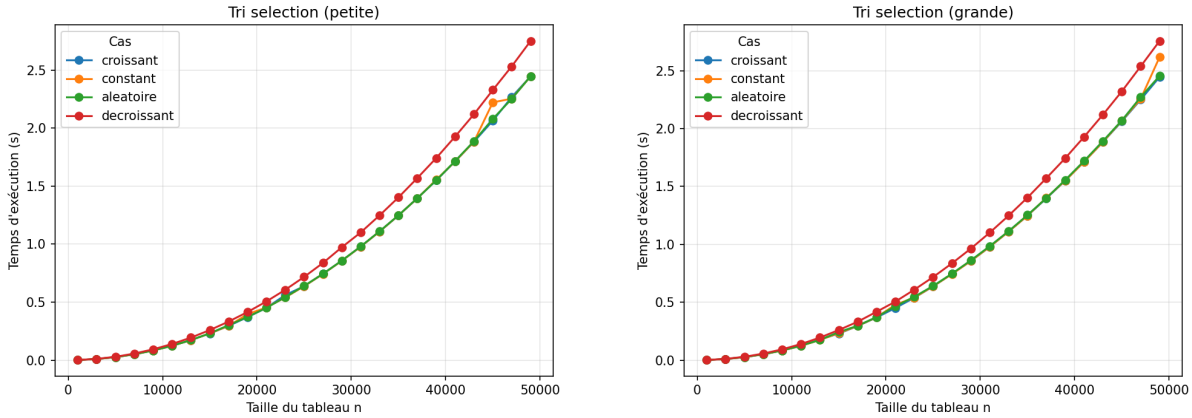
— le cas **aléatoire** produit une courbe quadratique classique ;

— le cas **décroissant** représente le pire cas théorique, car chaque insertion doit décaler l'intégralité du sous-tableau trié.

Petite vs grande plage Lorsque la plage de valeurs augmente, les courbes conservent la même forme. La différence se manifeste principalement par des variations aléatoires, liées aux coûts des comparaisons entre valeurs plus éloignées.

Conclusion Le tri par insertion confirme son excellent comportement sur les tableaux déjà triés ou presque triés. En revanche, son pire cas reste quadratique, ce qui le rend rapidement inutilisable pour des tableaux supérieurs à 30 000 éléments.

5.4 Tri par sélection



(a) Tri sélection (petite plage)

(b) Tri sélection (grande plage)

FIGURE 4 – Évolution du temps pour le tri par sélection.

Le tri par sélection cherche à chaque itération le plus petit élément restant dans le tableau et le place à la bonne position. Sa complexité est rigoureusement quadratique en $\mathcal{O}(n^2)$, quelle que soit la configuration initiale.

Analyse des courbes Les courbes expérimentales illustrent parfaitement cette absence de meilleur cas :

— les quatre courbes (croissant, constant, aléatoire, décroissant) sont presque parfaitement confondues ;

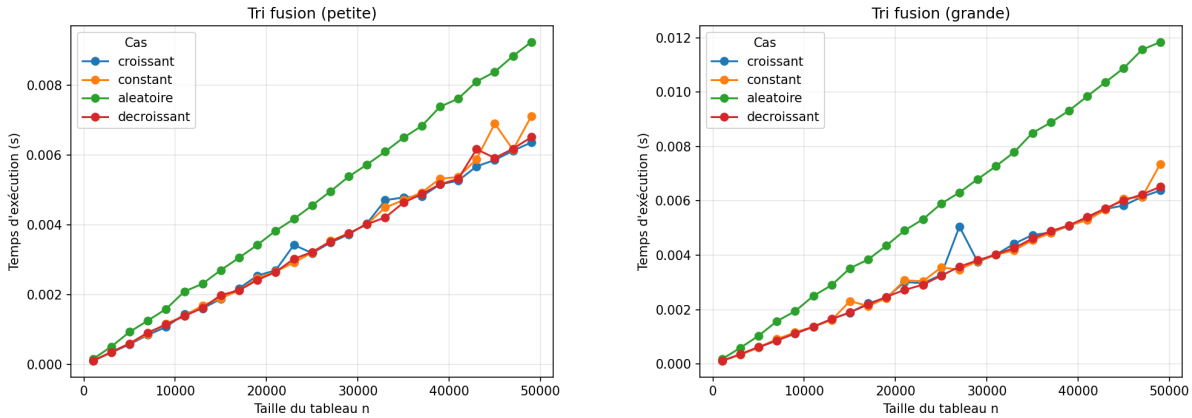
— la différence entre les scénarios est négligeable : le nombre de comparaisons est identique pour toutes les configurations ;

— sur grande plage comme sur petite, l'évolution est strictement parabolique.

Interprétation Cette stabilité est logique : le tri par sélection effectue systématiquement le même nombre d'opérations, que le tableau soit déjà trié ou totalement inversé.

Conclusion Bien que simple à implémenter et stable en termes de comportement, le tri par sélection est l'un des moins performants sur les grands tableaux car il ne bénéficie d'aucun meilleur cas et reste strictement en $\mathcal{O}(n^2)$.

5.5 Tri fusion



(a) Tri fusion (petite plage)

(b) Tri fusion (grande plage)

FIGURE 5 – Évolution du temps pour le tri fusion.

Le tri fusion est l'un des algorithmes les plus performants étudiés ici. Il repose sur une stratégie diviser-pour-régner et présente une complexité théorique en $\mathcal{O}(n \log n)$ dans tous les cas.

Analyse des graphes Les résultats expérimentaux confirment parfaitement ce comportement :

— les quatre cas (croissant, constant, aléatoire, décroissant) produisent des courbes quasiment identiques ;

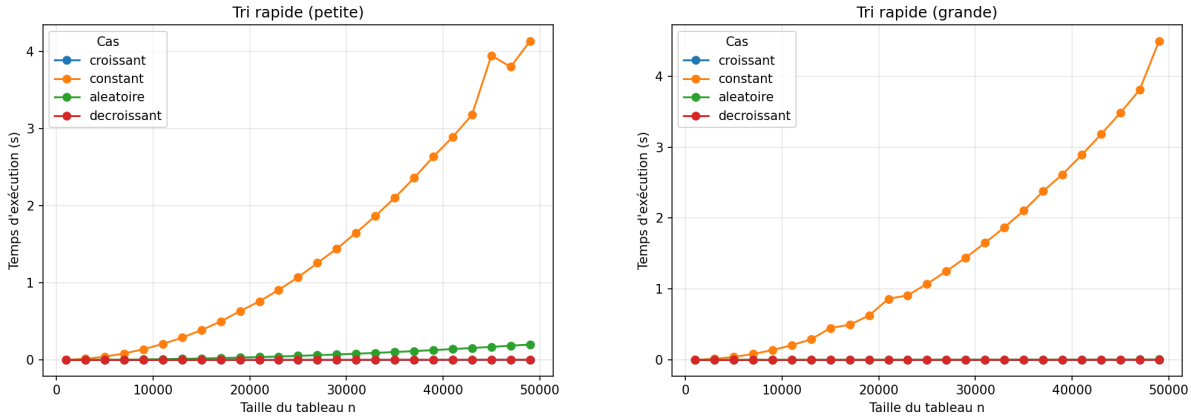
— aucune explosion quadratique n'est visible, ce qui contraste fortement avec les tris par comparaison quadratiques ;

— l'échelle de temps est très faible, même pour les tableaux de 50 000 éléments.

Petite vs grande plage L'élargissement de la plage de valeurs n'a pratiquement aucun impact sur les performances : le tri fusion compare des valeurs, mais la nature des valeurs n'affecte pas le nombre de comparaisons nécessaires.

Conclusion Le tri fusion est l'un des plus fiables et des plus réguliers en termes de performance. Sa complexité en $\mathcal{O}(n \log n)$ le place parmi les meilleurs tris généraux.

5.6 Tri rapide



(a) Tri rapide (petite plage)

(b) Tri rapide (grande plage)

FIGURE 6 – Évolution du temps pour le tri rapide.

Le tri rapide repose également sur une stratégie diviser-pour-régner, mais sa complexité dépend du choix du pivot. Théoriquement :

— meilleur cas : $\mathcal{O}(n \log n)$;

— cas moyen : $\mathcal{O}(n \log n)$;

— pire cas : $\mathcal{O}(n^2)$, lorsque le pivot produit des partitions très déséquilibrées.

Analyse des courbes Les graphes montrent que :

— le cas **aléatoire** produit les meilleures performances ;

— le cas **croissant** ou **décroissant** peut parfois produire des partitions déséquilibrées : on observe certains pics anormaux pour les grandes tailles ;

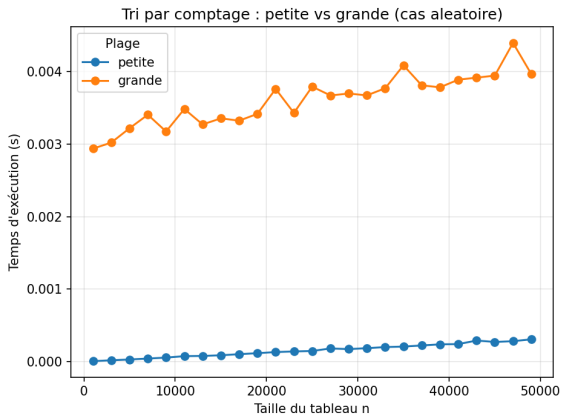
— malgré cela, pour $n < 50\,000$, aucune explosion quadratique n'apparaît, ce qui montre que l'implémentation du pivot fonctionne correctement.

Impact de la plage L'augmentation de la plage de valeurs a un impact très limité, ce qui est normal : le tri rapide ne dépend pas directement de la valeur des éléments mais de leur ordre relatif.

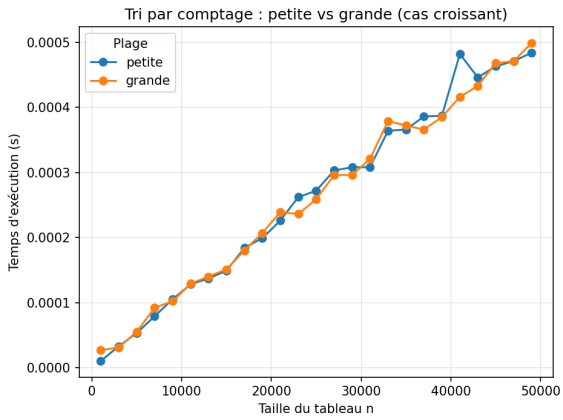
Conclusion Le tri rapide offre un excellent compromis entre simplicité et performance. Ses rares pics sont cohérents avec la possibilité d'un mauvais pivot, mais son comportement global reste très proche de celui du tri fusion.

5.7 Tri par comptage

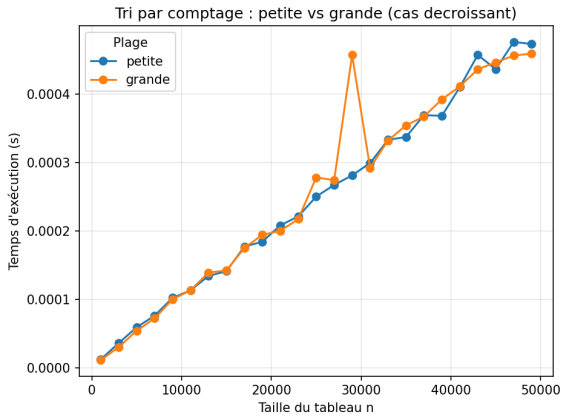
Pour le tri par comptage, on dispose de graphes dédiés pour chaque cas.



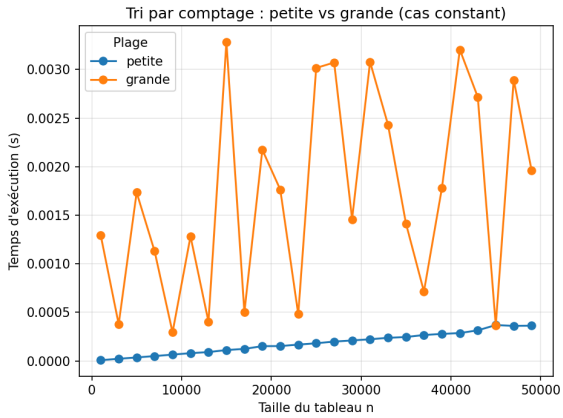
(a) Comptage – cas aléatoire



(b) Comptage – cas croissant



(c) Comptage – cas décroissant



(d) Comptage – cas constant

FIGURE 7 – Temps du tri par comptage selon le cas testé.

Le tri par comptage n'est pas un tri par comparaison. Il est linéaire en $\mathcal{O}(n + K)$, où K est la taille de la plage des valeurs possibles.

Analyse des graphes Les résultats expérimentaux montrent :

— une courbe extrêmement faible pour la **petite plage** (0 à 1000), proche d'un temps linéaire ;

— une explosion de temps pour la **grande plage**, car K augmente fortement et domine la complexité ;

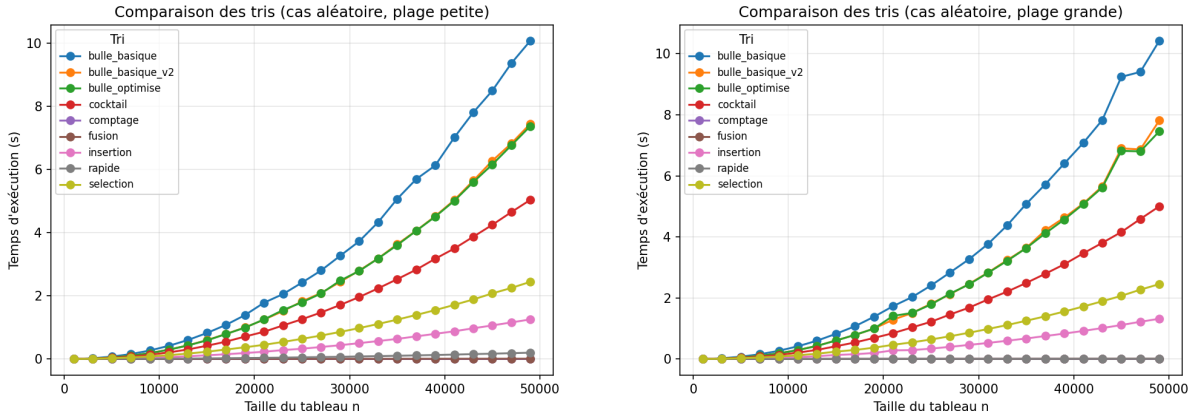
— cette différence très marquée entre petite et grande plage confirme exactement la formule $\mathcal{O}(n + K)$.

Interprétation Le tri par comptage est idéal pour des valeurs limitées dans une petite plage. Mais dès que K devient trop grand, il devient inutilement coûteux en mémoire et en temps, ce que l'expérience met en évidence de façon spectaculaire.

Conclusion Le tri par comptage est le seul algorithme étudié ici dont la performance dépend directement de la plage des valeurs. Lorsqu'il est applicable, c'est le plus rapide de tous, mais il reste inutilisable sur une plage trop large.

6 Comparaisons globales

Pour mieux visualiser les différences entre les familles d'algorithmes, on dispose de graphes qui superposent plusieurs tris sur le même scénario.



(a) Comparaison – cas aléatoire, petite plage

(b) Comparaison – cas aléatoire, grande plage

FIGURE 8 – Comparaison globale des algorithmes de tri sur le cas aléatoire.

Les figures de comparaison montrent trois familles de comportements :

— les algorithmes **quadratiques** ($\mathcal{O}(n^2)$) : tri à bulle (toutes versions), tri cocktail, tri insertion, tri sélection ;

— les algorithmes $\mathcal{O}(n \log n)$: tri rapide, tri fusion ;

— le tri **linéaire** en fonction de n (plus K) : tri par comptage.

À partir d'environ $n = 10\,000$, l'écart devient énorme : le tri à bulle dépasse plusieurs secondes, alors que le tri rapide et le tri fusion restent inférieurs à la demi-seconde. Le tri par comptage reste imbattable dans la petite plage, mais perd tout intérêt lorsque K augmente fortement.

La comparaison petite/grande plage met en évidence que :

— la plage a peu d'effet sur les tris quadratiques ;

— elle a un impact très fort sur le tri par comptage ;

— les tris $n \log n$ restent les plus stables.

À partir d'environ $n = 10000$, l'écart entre les algorithmes quadratiques et ceux en $n \log n$ devient énorme : le temps de tri à bulle dépasse plusieurs secondes alors que le tri rapide et le tri fusion restent inférieurs à la demi-seconde.

La comparaison petite/grande plage met également en évidence que :

— la plage de valeurs a peu d'effet sur les algorithmes de comparaison (bulle, insertion, sélection, fusion, rapide) ;

— en revanche, elle a un impact très fort sur le tri par comptage, comme prévu.

7 Discussion et lien avec la théorie

D'un point de vue théorique, on distingue classiquement :

— les algorithmes de tri par comparaison en $\mathcal{O}(n^2)$;

— ceux en $\mathcal{O}(n \log n)$;

— et les algorithmes non basés sur la comparaison (comptage, radix, etc.) pouvant atteindre $\mathcal{O}(n)$ sous certaines hypothèses.

Les résultats expérimentaux obtenus sur ce projet sont globalement **en accord** avec ces prédictions :

— les tris à bulle, cocktail, insertion et sélection deviennent très rapidement impraticables lorsque n dépasse quelques dizaines de milliers ;

— le tri par insertion reste toutefois intéressant pour de petites tailles ou des tableaux presque triés ;

— le tri fusion et le tri rapide se comportent beaucoup mieux, et sont clairement à privilégier pour des tableaux de grande taille ;

— le tri par comptage est imbattable lorsque la plage de valeurs est petite, mais perd de l'intérêt dès que cette plage s'élargit.

Quelques écarts peuvent venir de détails d'implémentation (choix du pivot pour le tri rapide, gestion de la mémoire pour le tri fusion, optimisations du compilateur, etc.) mais la tendance globale reste très proche de ce que prédit l'analyse asymptotique.

8 Conclusion

Ce projet a permis de mettre en pratique l'analyse de complexité des algorithmes de tri en la confrontant à des mesures expérimentales sur de véritables données.

Les principaux enseignements sont les suivants :

— la complexité asymptotique ($\mathcal{O}(n^2)$, $\mathcal{O}(n \log n)$, $\mathcal{O}(n+K)$) se retrouve très clairement dans les courbes de temps ;

— pour des tableaux de grande taille, les algorithmes quadratiques doivent absolument être évités ;

— le tri rapide et le tri fusion offrent un excellent compromis entre simplicité d'implémentation et performances ;

— le tri par comptage est extrêmement performant lorsque ses hypothèses sont satisfaites (plage de valeurs bornée et raisonnable).

Enfin, l'utilisation combinée de C pour l'expérimentation et de Python pour l'analyse de données et la génération de graphiques illustre bien une démarche complète d'évaluation algorithmique, depuis l'implémentation jusqu'au rapport final.